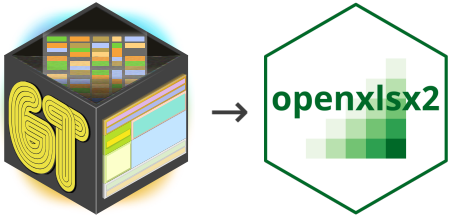
gtxlsx



gt tables, and any HTML table,
in an Excel worksheet

Jan Marvin Garbuszus

gt

pizzaplace	
counts by type and size	
	sold
classic	
S	6,139
M	4,112
L	4,057
total	14,308
veggie	
S	2,663
M	3,583
L	5,403
total	11,649
all	25,957
Source: the pizzaplace data set.	



xlsx

	A	B	C
1			
2		pizzaplace	
3		counts by type and size	
4			sold
5		classic	
6		S	6,139
7		M	4,112
8		L	4,057
9		total	14,308
10		veggie	
11		S	2,663
12		M	3,583
13		L	5,403
14		total	11,649
15		all	25,957
16		Source: the pizzaplace data set.	

```
wb_add_gt(wb, tbl)
```

Heading, row groups, stub, summaries, source note, fills, borders: all in the sheet.

Two packages, no bridge

Copy and paste loses the numbers.

A screenshot loses the structure.

gtxlsx takes a gt table and writes it into a worksheet.

It also takes any HTML table.

gt builds the table you show people

```
gt(pizza, rowname_col = "size",
   groupname_col = "type") |>
  fmt_integer(columns = sold) |>
  summary_rows(columns = sold,
    fns = list(total = ~ sum(.x))) |>
  grand_summary_rows(columns = sold,
    fns = list(all = ~ sum(.x))) |>
  tab_header(title = "pizzaplace") |>
  tab_source_note("Source: ...") |>
  opt_stylize(style = 3, color = "blue")
```

pizzaplace

counts by type and size

	sold
classic	
S	6,139
M	4,112
L	4,057
total	14,308
veggie	
S	2,663
M	3,583
L	5,403
total	11,649
all	25,957

Source: the pizzaplace data set.

openxlsx2 writes the worksheet

```
wb <- wb_workbook()$add_worksheet("data")
wb$add_data(x = head(mtcars), start_col = 2, start_row = 2)
wb$add_fill(dims = "B2:L2", color = wb_color(hex = "FF204060"))
wb$add_font(dims = "B2:L2", bold = TRUE, color = wb_color(hex = "FFFFFFFF"))
```

	A	B	C	D	E	F	G	H	I	J	K	L
1												
2		mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
3		21	6	160	110	3.9	2.62	16.46	0	1	4	4
4		21	6	160	110	3.9	2.875	17.02	0	1	4	4
5		22.8	4	108	93	3.85	2.32	18.61	1	1	4	1
6		21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
7		18.7	8	360	175	3.15	3.44	17.02	0	0	3	2
8		18.1	6	225	105	2.76	3.46	20.22	1	0	3	1

It has no idea what a gt table is.

One function

```
library(gt); library(openxlsx2); library(gtxlsx)
wb <- wb_workbook()$add_worksheet("report")
wb <- wb_add_gt(wb, gt(head(exibble, 4)), dims = "B2")
```

gt

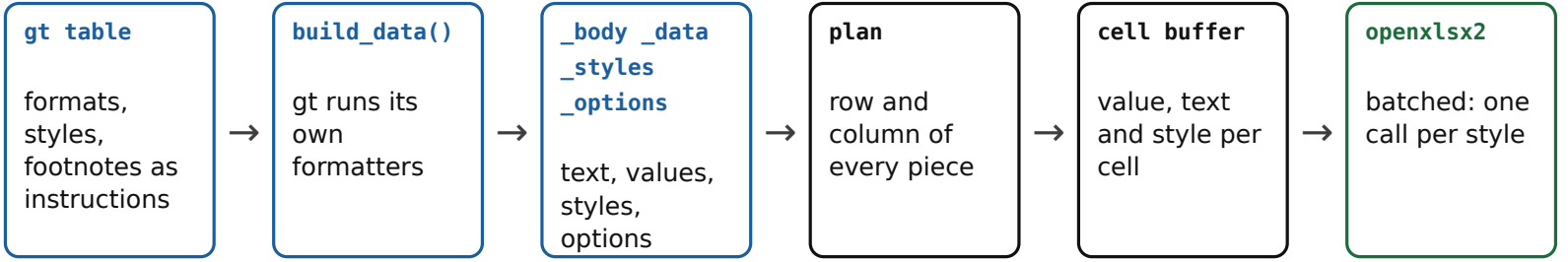
num	char	fctr	date	time	datetime	currency	row	group
0.1111	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.95	row_1	grp_a
2.2220	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.95	row_2	grp_a
33.3300	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.39	row_3	grp_a
444.4000	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.00	row_4	grp_a

xlsx

num	char	fctr	date	time	datetime	currency	row	group
0.1111	apricot	one	2015-01-15	13:35	2018-01-01 02:22	49.95	row_1	grp_a
2.2220	banana	two	2015-02-15	14:40	2018-02-02 14:33	17.95	row_2	grp_a
33.3300	coconut	three	2015-03-15	15:45	2018-03-03 03:44	1.39	row_3	grp_a
444.4000	durian	four	2015-04-15	16:50	2018-04-04 15:55	65100.00	row_4	grp_a

▶ live: example 1

What happens in between



gt does the formatting. **gtxlsx** places and translates. **openxlsx2** writes.

What gt hands over

```
tbl <- gt(data.frame(a = c("plain", "bold"),
                     n = c(1.5, 2))) |>
  fmt_number(columns = n, decimals = 1) |>
  tab_footnote("note",
    cells_body(columns = a, rows = 1)) |>
  tab_style(cell_text(weight = "bold"),
    cells_body(columns = n, rows = 2))

built <- gt:::build_data(tbl, "html")
built[["_data"]]$n
built[["_body"]]$n
built[["_body"]]$a[1]
s <- as.data.frame(built[["_styles"]])
s[, c("locname", "colname", "rownum")]
```

```
[1] 1.5 2.0
[1] "1.5" "2.0"
[1] "plain<span class=\"gt_footnote_marks\"
 style=\"white-space:nowrap;font-style:italic;
 font-weight:normal;line-height:0;\">
 <sup>1</sup></span>"
  locname colname rownum
1    data       n      2
```

Values and printed text side by side. A bold from **`tab_style()`** is a style entry beside the text, not a **`<b>`** tag in it.

▶ live: example 12

Markup in a cell becomes rich text

`<span style="font-style:italic;…">`	italic
`<sup>`	italic, superscript
`1`	run: **“1”**, italic, superscript
`</sup></span>`	back to plain
`1`	run: **“1”**, plain

One regex finds the tags, a style stack follows them.

	A	B
1		
2		a
3		[1] 1
4		2
5		[1] note

two runs, one cell

The cell holds the number

gt prints	**€1,204.50**	**44.6%**	**1.2K**
stored value	**1204.5**	**0.446**	**1200**
a valid rounding?	✓	✓	✗
the cell gets	**1204.5**	**0.446**	**text: 1.2K**
number format	**"€"#,##0.00**	**0.0%**	**none**

The printed text has to be a rounding of the stored value, within half a unit of the last digit.

▶ live: example 4

What it looks like

gt

money	share	count	when
€1,204.50	44.6%	12,000	31 January 2026
€700.00	35.2%	4,300	30 June 2026
€56.00	20.1%	987	24 December 2026

xlsx

	A	B	C	D	E
1					
2		money	share	count	when
3		€1,204.50	44.6%	12,000	31 January 2026
4		-€700.00	35.2%	4,300	30 June 2026
5		€56.00	20.1%	987	24 December 2026

Dates are written as text in 0.4.0. The development version converts them to spreadsheet dates.

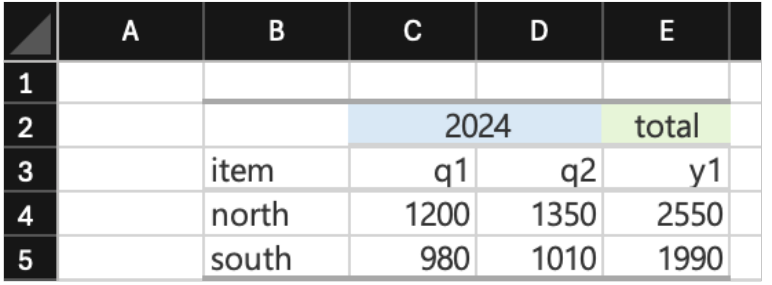
Spanners become merged cells

gt

	2024		total
item	q1	q2	y1
north	1200	1350	2550
south	980	1010	1990

→

xlsx



	A	B	C	D	E
1					
2			2024		total
3		item	q1	q2	y1
4		north	1200	1350	2550
5		south	980	1010	1990

Spanners are merged cells. Each keeps its own fill.

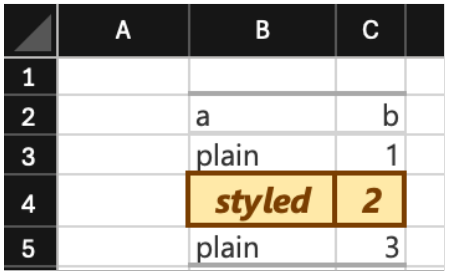
▶ live: example 3

Everything you can do to one cell

gt

a	b
plain	1
styled	***2***
plain	3

xlsx



Fill, font, size, color, alignment and a border. Any location: body, stub, labels, spanners, summaries, notes.

▶ live: example 2

A report, not just a grid

	A	B	C
1			
2		pizzaplace	
3		counts by type and size	
4			sold
5		classic	
6		S	6,139
7		M	4,112
8		L	4,057
9		total	14,308
10		veggie	
11		S	2,663
12		M	3,583
13		L	5,403
14		total	11,649
15		all	25,957
16		Source: the pizzaplace data set.	

heading and subtitle, merged

column label, fill from the theme

row group, its own heading row

stub

summary per group

grand summary

source note, merged

▶ live: example 6

It does not have to be gt

```
html <- knitr::kable(head(mtcars[, 1:4]), format = "html")

wb <- wb_workbook()$add_worksheet("kable")
wb <- wb_add_html(wb, html, dims = "B2")
```

html

	mpg	cyl	disp	hp
Mazda RX4	21.0	6	160	110
Mazda RX4 Wag	21.0	6	160	110
Datsun 710	22.8	4	108	93
Hornet 4 Drive	21.4	6	258	110
Hornet Sportabout	18.7	8	360	175
Valiant	18.1	6	225	105

→

xlsx

	A	B	C	D	E	F
1						
2			mpg	cyl	disp	hp
3		Mazda RX4	21.0	6	160	110
4		Mazda RX4 Wag	21.0	6	160	110
5		Datsun 710	22.8	4	108	93
6		Hornet 4 Drive	21.4	6	258	110
7		Hornet Sportabout	18.7	8	360	175
8		Valiant	18.1	6	225	105

▶ live: example 8

A table straight from a web page

```r
library(rvest)

url <- "https://en.wikipedia.org/wiki/List_of_circulating_currencies"
tab <- read_html(url) |> html_elements("table")

wb <- wb_workbook() |> wb_add_worksheet() |>
  wb_add_html(x = tab[2], dims = "A1", freeze = TRUE)
```

Links stay clickable.

`freeze = TRUE` keeps the header rows in view.

`which` picks a table when you pass a whole page.

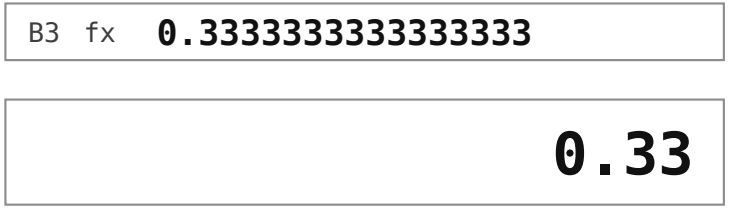
▶ live: example 10

Switches worth knowing

`features = FALSE`	values only
`features = c("numfmt", "merge")`	choose what is written besides values
`freeze = TRUE`	heading and stub stay in view
`row_heights = "gt"`	from gt's padding, text centered
`gap = 2`	blank rows between the tables of a gt_group

▶ live: example 11

The cell knows more than the table showed



formula bar above, the cell below

gt prints a rounded string.

gtxlsx writes the value, and a format that looks the same.

If the printed value is the deliverable, round the data before gt sees it.

▶ live: example 9a

R rounds half to even, Excel away from zero

value	gt shows	Excel displays
0.5	0	1
1.5	2	2
2.5	2	3
3.5	4	4

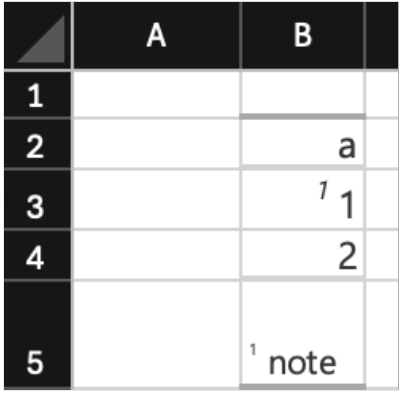
Both round correctly, by different conventions. Rare with measured data, common in examples that end in .5.

▶ live: example 9b

Two things change a column

Footnote marks

a
[1] 1
2
[1] note

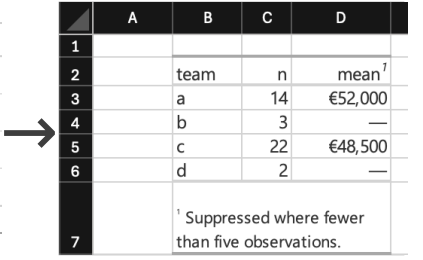


Excel has no footnote object. The mark is text inside the cell, so the whole column reads back as text.

Censored cells

team	n	mean[1]
a	14	€52,000
b	3	—
c	22	€48,500
d	2	—

[1] Suppressed where fewer than five observations.



Masked in gt means masked in the sheet. One text cell makes the whole column text.

▶ live: example 5, 9c

What does not carry over

Pictures

`fmt_image()` and **`cols_nanoplot()`** leave the cell empty.

`fmt_icon()` and **`fmt_flag()`** keep their label text.

Interactivity

`opt_interactive()` and **`opt_css()`** have no effect.

One configures a widget, the other a browser. A worksheet has neither.

Approximations

Row heights are opt-in. Excel does not fit merged rows, so a long note can be cut off; the development version estimates the height.

On the HTML path, **`:has()`** and **`::before`** rules are skipped, not applied.

Everything here fails by leaving something out, which you can see.

Related work

gt-pixel-art Melissa Van Bussel

Her entry to Posit's 2025 Table Contest reads an xlsx with openxlsx2 and draws it as a gt table. It led me to rework my first idea, and **`wb_to_gt()`** goes the same way.
`github.com/melissavanbussel/gt-pixel-art`

forgts Luis D. Verde Arregoitia

A spreadsheet in, a gt table out, with fills, fonts and borders kept. Luis has written about gtxlsx as the way back. I did not know forgts while building gtxlsx.
`github.com/luisDVA/forgts`

gtopenxlsx Yann Say

Exports gt tables to xlsx, for the older openxlsx package. I came across it after my first draft was done (openxlsx2 discussion #1208).
`github.com/yannsay/gtopenxlsx`

Where it stands

```
install.packages("gtxlsx")
```

openxlsx2

the only hard dependency

gt ≥ 0.8.0

optional, in Suggests

R ≥ 3.6

no newer syntax

Tested on gt 0.8.0, 1.3.0 and the current development version.

wb_add_html() needs nothing but openxlsx2.

gtxlsx

github.com/JanMarvin/gtxlsx

wb_add_gt(wb, tbl)	a gt table, or a gt_group, into a worksheet
wb_add_html(wb, x)	any HTML table, with only openxlsx2
wb_add_lt(wb, tbl)	lt tables